

第一个月详细指导书

AI-assisted LQCD Meson DA Analysis Agent

项目训练材料：面向研究生的学习、分析与搭建路线

版本：Month 1 Detailed Guide

目录

1	第一个月的定位	4
2	第一个月的总体交付物	4
3	学生每天的工作方式	5
3.1	Step 1: 先写当天小目标	6
3.2	Step 2: 让 Codex 做小而具体的任务	6
3.3	Step 3: 学生自己运行和检查	6
3.4	Step 4: 用 traceback 让 Codex 修复	6
3.5	Step 5: Git 小步提交	7
4	Week 1: 真实 example 与统计物理基础	7
4.1	Week 1 的学习-练习循环	7
4.1.1	建议阅读路线	7
4.2	Week 1 与后续分析的衔接	8
4.2.1	当前 example 的真实主线	8
4.2.2	两点 correlator 的物理模型	9
4.2.3	effective mass 的含义	10
4.2.4	bootstrap、binning 和协方差	10
4.2.5	最小 χ^2 非线性拟合	12
4.2.6	bootstrap-centered 的中心值和误差	13
4.2.7	从 0-effective-mass 到 1-c2pt-fit	14
4.2.8	0-effective-mass: 把概念变成图	15
4.2.9	如何读 effective mass 图	17
4.2.10	Week 1 的练习与检查	17
4.2.11	DA ratio 的最小物理图像	18
4.2.12	1-c2pt-fit: 从 plateau 到参数提取	19
4.2.13	两点拟合的窗口、状态数和先验选择	21

4.2.14	DA ratio 的窗口和诊断逻辑	23
4.2.15	Week 1 和后续 example 的连接	24
4.3	如何运行当前 example: 先看原则, 细节放到后面的实操部分	24
4.4	Week 1 的实操主线: 环境、代码理解与 golden example 复现	24
4.4.1	Week 1 目标	24
4.4.2	Week 1 的学习-练习循环	25
4.4.3	Week 1 实操主线: 先 notebook, 再 input file	25
4.4.4	Week 1 每日任务单	26
5	Week 2: Config-driven CLI 与 Pydantic schema	33
5.1	Week 2 目标	33
5.2	Week 2 学习内容	33
5.3	Week 2 任务 1: 定义 config schema	34
5.4	Week 2 任务 2: 定义 result schema	35
5.5	Week 2 任务 3: 实现最小 CLI	36
5.6	Week 2 命令示例	37
5.7	Week 2 常见错误	37
5.8	Week 2 导师检查问题	38
5.9	Week 2 任务 4: 记录 Codex 使用日志	38
6	Week 3: 数据检查、最小测试体系与输出规范	38
6.1	Week 3 目标	38
6.2	Week 3 学习内容	38
6.3	Week 3 任务 1: data audit 模块 (Data Integrity Check / Correlator Analyst)	39
6.4	Week 3 任务 2: 最小 pytest 体系	40
6.5	Week 3 任务 2.5: 生成 correlator_diagnostics.json	40
6.6	Week 3 任务 3: 输出规范	41
6.6.1	manifest.json 应包含	41
6.6.2	result.json 应包含	41
6.6.3	summary.csv 应包含	41
6.7	Week 3 命令示例	42
6.8	Week 3 常见错误	42
6.9	Week 3 导师检查问题	42
6.10	Week 3 验收标准	42
7	Week 4: Month 1 集成、最小 Analysis Engineer 雏形与展示	43
7.1	Week 4 目标	43
7.2	Week 4 学习内容	43
7.3	Week 4 任务 1: 定义 Analysis Engineer task schema	43
7.4	Week 4 任务 2: 最小 Analysis Engineer wrapper	44
7.5	Week 4 任务 3: Month 1 final report	45

7.6	Week 4 命令示例	46
7.7	Week 4 常见错误	46
7.8	Week 4 导师检查问题	46
7.9	Week 4 任务 4: Month 1 demo	46
8	第一个月导师验收清单	47
8.1	复现性	47
8.2	代码质量	47
8.3	测试	47
8.4	学生理解	47
9	第一个月常见错误	48
9.1	错误 1: 一开始就让 Codex 写完整系统	48
9.2	错误 2: 只跑 notebook, 不记录命令	48
9.3	错误 3: 没有 schema	48
9.4	错误 4: 没有 tests	48
9.5	错误 5: 把 Analysis Engineer 当成物理判断者	48
9.6	错误 6: 输出没有 provenance	48
10	Month 2 的衔接	48
11	附录: 第一个月推荐 Codex Prompt 模板	49
11.1	通用模板	49
11.2	修 bug 模板	49
11.3	让 Codex 加测试的模板	50
12	第一个月最终标准	50

1 第一个月的定位

第一个月的核心目标不是立刻做出完整的高级 agent 系统，而是建立一个**可靠、可运行、可测试、可复现、可审计**的基础版本。学生需要通过第一个月完成从“会运行别人代码”到“能把科研分析流程工程化”的转变。

第一个月结束时，学生应该能够做到：

1. 复现一个已经准备好的 golden example，例如 `examples/l64c64a076_m140` 或导师指定的 meson DA 分析样例；
2. 说清楚这个样例的输入、输出、中间文件、关键图和主要物理检查（尤其是 effective mass plateau 诊断，这对应后续 Correlator Analyst Agent 的职责）；
3. 把原本 notebook-style 或手动运行的流程整理成 config-driven Python CLI；
4. 使用 Pydantic 定义 config schema 和 result schema；
5. 使用 pytest 对关键模块做基本测试；
6. 使用 Codex 辅助写代码、修 bug、补 tests，但不盲目信任 Codex；
7. 用 Git 做小步提交，保证每一步可以回退；
8. 输出一个基础的 run manifest、result.json、summary.csv、plots 和 run.log；
9. 写出第一版项目说明和复现实验报告。

第一个月不要求学生完全掌握 Snakemake 和 LangGraph，但需要理解它们后续的定位：

- Snakemake 后续用于管理真实分析流程的文件依赖和增量重跑；
- LangGraph 后续用于组织多 agent 状态流转、human approval、targeted rerun 和 memory update；
- 第一个月的重点是先把每个基本模块、输入输出和测试做清楚，否则后续 workflow engine 和 agent orchestration 会变成空架子。

2 第一个月的总体交付物

第一个月结束时，学生应提交以下内容：

```
lqcd-meson-da-agent/  
  README.md  
  docs/  
    month1_reproduction_report.md  
    data_flow_map.md  
    codex_usage_log.md
```

```
configs/  
  golden_example.yaml  
src/  
  lqcd_agent/  
    __init__.py  
    schema/  
      analysis_config.py  
      analysis_result.py  
    audit/  
      data_audit.py  
    cli/  
      run_golden.py  
    utils/  
      logging_utils.py  
      io_utils.py  
tests/  
  test_schema.py  
  test_data_audit.py  
  test_cli_smoke.py  
outputs/  
  golden_example/  
    manifest.json  
    result.json  
    summary.csv  
  plots/  
  run.log
```

如果当前项目已有不同目录结构，学生不需要强行照搬上述结构，但必须满足以下原则：

1. 输入配置集中管理；
2. 原始数据不被修改；
3. 输出目录有明确 run id；
4. 每次运行留下 manifest 和 log；
5. 关键结果以 JSON/CSV 结构化保存；
6. 至少有最小 pytest 测试；
7. README 能让另一个人照着复现。

3 学生每天的工作方式

学生每天都应按照下面的循环工作。

3.1 Step 1: 先写当天小目标

不要一上来让 Codex “帮我做完整项目”。每天先写一个小目标，例如：

Today goal:

```
Run the golden example from a clean environment, record all required inputs and
outputs, and write a data-flow map.
```

或者：

Today goal:

```
Add a Pydantic schema for the golden example config and write tests for missing fields
and invalid paths.
```

3.2 Step 2: 让 Codex 做小而具体的任务

Codex prompt 必须包含：

- 当前项目背景；
- 要修改哪些文件；
- 输入和输出是什么；
- 不允许修改什么；
- 需要添加哪些 tests；
- 完成后如何运行检查。

3.3 Step 3: 学生自己运行和检查

Codex 写完后，学生必须自己运行：

```
pytest
python -m lqcd_agent.cli.run_golden --config configs/golden_example.yaml
```

同时检查输出文件是否真的生成，图是否能打开，JSON/CSV 是否符合预期。

3.4 Step 4: 用 traceback 让 Codex 修复

如果报错，不要简单说“修一下”。应该把 traceback、运行命令、预期行为和实际行为给 Codex。

The command failed:

```
python -m lqcd_agent.cli.run_golden --config configs/golden_example.yaml
```

Traceback:

```
[paste traceback]
```

Expected behavior:

The CLI should load the config, run the golden example, and write manifest.json.

Please fix only the cause of this error. Do not refactor unrelated files.

Also add a regression test if appropriate.

3.5 Step 5: Git 小步提交

每个完成的小任务都应该提交一次。

```
git status
git diff
git add <changed files>
git commit -m "Add config schema for golden example"
```

不要把一周所有修改堆到一个 commit。

4 Week 1: 真实 example 与统计物理基础

这一节是 Month 1 的核心背景。它把真实 example 的数据流、统计直觉和代码结构串在一起，目标不是先背概念，而是先把“看图、动手、复现”这一条最小闭环跑通。

Week 1 的目标不是让学生“看懂结果”，而是让学生建立最基本的科研判断方式。对于两点函数分析来说，最重要的第一步不是拟合，而是学会读图：知道什么叫 plateau，知道什么时候噪声开始压过信号，知道为什么不同动量的 correlator 不能直接用同一套直觉去解释。

从教学上说，Week 1 的前半段就是一个最小闭环：

1. 读入一组两点 correlator；
2. 只计算 effective mass；
3. 只画图，不做复杂拟合；
4. 从图上回答“这里是不是已经接近单态区”；
5. 把观察到的 plateau 和噪声结构带到后面的 1-c2pt-fit。

如果这一步没有学透，后面的 two-point fit 和 DA ratio 会变成纯粹的黑箱操作。

4.1 Week 1 的学习-练习循环

4.1.1 建议阅读路线

如果你是第一次接触这套分析，建议按下面的顺序读本章，而不是从头到尾随便翻。这里的原则是：先建立统计和物理直觉，再做最小练习，最后才进入真正的拟合。

1. 先看 **当前 example 的真实主线**，理解数据从哪里来、要去哪里；

2. 接着看 **两点 correlator 的物理模型**，弄清楚 A_n 和 E_n 是什么，以及为什么拟合参数要满足物理约束；
3. 然后看 **bootstrap、binning、协方差、 χ^2** ，理解误差条、相关性和拟合优度是怎么从样本里来的；
4. 再打开 **0-effective-mass**，把前面学到的概念立刻用在图上，先建立 plateau、噪声和 bootstrap 的直觉；
5. 再看 **1-c2pt-fit**，把 effective mass 的直觉转成真正的多态拟合窗口选择；
6. 之后看 **DA ratio 的最小物理图像**，把两点拟合和后面的矩阵元提取连起来；
7. 最后看 **notebook 与 input file 的运行方式**，把理解过的流程固化成可复现输入。

这个顺序的原则很简单：先看“数据流”，再看“模型”，再看“统计”，然后立刻用 0-effective-mass 把概念落到图上，最后再把图上的直觉带进真正的拟合。

更适合初学者的方式，是把每一个知识点都立刻对应到一个动作上：

1. 先看当前 example 的主线和两点 correlator 的模型；
2. 再理解 bootstrap、binning、协方差和 χ^2 的统计含义；
3. 立刻跑 0-effective-mass 的 notebook，把图画出来；
4. 立刻再跑 input file，把 notebook 里的理解冻结成可复现输入；
5. 立刻写下自己观察到的 plateau、噪声和误差变化；
6. 再把这些观察带到 1-c2pt-fit 里去选窗口。

4.2 Week 1 与后续分析的衔接

4.2.1 当前 example 的真实主线

本仓库当前的实际 example 是 `examples/164c64a076_m140/`。它是一条已经裁剪成 DA-only 目标的真实分析链，包含 effective mass、两点拟合、DA bare matrix element、normalization 和 Fourier 变换。

最常见的分析入口是：

```
python -m lqcd_analysis.cli 2pt-effective-mass ...
python -m lqcd_analysis.cli 2pt-nstate-fit ...
python -m lqcd_analysis.cli da-nstate-fit ...
python -m lqcd_analysis.cli da-normalize ...
python -m lqcd_analysis.cli da-fourier ...
```

学生至少要理解这四段之间的依赖关系：

阶段	结果层级	关键物理问题
0-effective-mass	effective mass 和图	plateau、噪声、bootstrap 误差是否合理
1-c2pt-fit	两点 correlator 拟合	能量、overlap、 χ^2/dof 是否稳定
2-bm	DA bare matrix element	是否已把 ratio 中的矩阵元提取出来
3-normalize	归一化后的中间量	只保留 mode3 路径，检查归一化是否合理
4-FT	Fourier 变换结果	最终分布的形状和物理趋势是否正常

当前 example 的数据已经被裁剪成一条更窄但更清楚的 DA-only 主线：输入里只保留 `b_X`、`OT5` 和 `bT = 0`，同时删掉了 `b_Y` 和 `OT5`，而下游 analysis 也只保留 `mode3` 路径。学生在读这一章时，只需要知道这套 example 的数据长什么样、为什么只剩这些分支，以及这些分支在后续分析里分别扮演什么角色，不需要再记一串 bullet。

这份 Month 1 章节已经把 two-point fit 和 DA ratio-fit 的关键物理、统计和代码逻辑合并在一起了，所以学生不需要再翻独立的旧 guide。真正需要反复看的，是当前 example 的目录结构和输入文件约定。

4.2.2 两点 correlator 的物理模型

对任意插入算符 O ，两点函数的定义是

$$C(t) = \langle O(t) O^\dagger(0) \rangle. \quad (1)$$

在有限时长 N_t 的格点上，当前实现支持三种模型：

$$C_{\text{normal}}(t) = \sum_{n=0}^{N_{\text{state}}-1} A_n e^{-E_n t}, \quad (2)$$

$$C_{\text{symmetric}}(t) = \sum_{n=0}^{N_{\text{state}}-1} A_n \left(e^{-E_n t} + e^{-E_n (N_t - t)} \right), \quad (3)$$

$$C_{\text{antisymmetric}}(t) = \sum_{n=0}^{N_{\text{state}}-1} A_n \left(e^{-E_n t} - e^{-E_n (N_t - t)} \right). \quad (4)$$

这里的 A_n 是 overlap/amplitude, E_n 是能量。代码中不会直接拟合任意无序参数，而是通过对数与 gap 参数化强制：

- $A_n > 0$;
- $E_0 > 0$;
- $E_{n+1} > E_n$ 。

所以拟合不是“猜一个数”，而是在一个满足物理约束的参数空间里做非线性最小二乘。

4.2.3 effective mass 的含义

对于 `model = normal`, effective mass 定义为

$$m_{\text{eff}}(t) = \log \frac{C(t)}{C(t+1)}. \quad (5)$$

对于 periodic / antiperiodic 的情况, 代码会数值求解 cosh / sinh 版本的方程:

$$\frac{C(t)}{C(t+1)} = \frac{\cosh[m_{\text{eff}}(t+1 - N_t/2)]}{\cosh} [m_{\text{eff}}(t - N_t/2)] \quad (\text{symmetric}), \quad (6)$$

$$\frac{C(t)}{C(t+1)} = \frac{\sinh[m_{\text{eff}}(t+1 - N_t/2)]}{\sinh} [m_{\text{eff}}(t - N_t/2)] \quad (\text{antisymmetric}). \quad (7)$$

effective mass 的作用不是最终物理结论, 而是 plateau 诊断: 它帮助判断拟合窗口是否已经进入近似单态区。

4.2.4 bootstrap、binning 和协方差

先回答一个最根本的问题: **为什么这里要 bootstrap sampling?**

因为格点数据里的“误差”不是凭空给出的, 而是要从一批带噪声的 configuration 里估计出来。我们关心的不是某一次测量值, 而是“如果重新抽一批样本, 均值、误差条和拟合参数会怎么波动”。bootstrap 正是在做这件事。

当前代码的统计顺序是:

1. 先对原始 configuration 沿第一维做 binning, 把相邻配置压成 block;
2. 再从 binned blocks 中有放回抽样;
3. 每个 bootstrap sample 都是一条新的平均 correlator;
4. 用 bootstrap sample 的分布估计误差、协方差和参数不确定性。

这一步背后的统计含义是: 我们并不把单个 configuration 当成“真值”, 而是把一批经过 binning 之后的 block 当成经验样本, 再用 bootstrap 去逼近“重复做一次测量时, 结果会怎样波动”。对于格点数据来说, 这一点尤其重要, 因为相邻 configuration 可能存在自相关。如果不先 binning, 就可能把相关样本误当成独立样本, 从而把误差条估得过于乐观。

把这件事翻译成可操作的算法, 就是:

1. 从原始 correlator 读入一个形状类似 `(n_cfg, Nt)` 的数组;
2. 按 `binsize` 在 configuration 维度上分块平均;
3. 每次 bootstrap resample 都从这些 blocks 里“有放回”抽取 `n_draws` 个 block;
4. 对抽到的 block 再求平均, 得到一条 bootstrap mean correlator;
5. 重复很多次, 最后得到 bootstrap 分布。

对学生来说，bootstrap、binning 和协方差分别回答的是三件不同的事：

- **binning** 负责降低自相关的影响；
- **bootstrap** 负责把样本波动传播到最终物理量；
- **covariance** 负责告诉你不同时间点是不是一起涨、一起跌。

所以这一步不是“为了多算几遍”，而是为了把统计不确定性和时间相关结构都显式地保留下来。

如果再往代码里看一层，真实顺序就是 `apply_fold_t` 先利用时间反向对称性把 correlator 整理得更平滑，然后 `bin_correlators` 压低自相关，接着 `bootstrap_correlator_means` 生成重采样平均值，最后这些 bootstrap means 被送进 `effective_mass_with_bootstrap`、协方差估计和拟合模块。

如果第 b 个 bootstrap 样本的 correlator 是 $C^{(b)}(t)$ ，那么均值可写为

$$\bar{C}(t) = \frac{1}{N_{\text{boot}}} \sum_{b=1}^{N_{\text{boot}}} C^{(b)}(t). \quad (8)$$

协方差矩阵由 bootstrap 样本估计：

$$\text{Cov}_{ij} = \frac{1}{N_{\text{boot}} - 1} \sum_{b=1}^{N_{\text{boot}}} \left(C_i^{(b)} - \bar{C}_i \right) \left(C_j^{(b)} - \bar{C}_j \right). \quad (9)$$

如果协方差矩阵不稳定，代码会做 shrinkage：

$$C_\lambda = (1 - \lambda)C + \lambda \text{diag}(C). \quad (10)$$

其中 $\lambda = 0$ 保留完整协方差， $\lambda = 1$ 只保留对角项。

在本仓库里，bootstrap 的实现顺序是固定的。先把 correlator 组织成 `(n_cfg, Nt)` 的数组；然后按 `binsize` 沿第一个维度做 binning，得到更稳定的 block；接着用 `bootstrap_indices` 在这些 block 上做有放回抽样；最后用 `bootstrap_means` 计算每个 resample 的平均 correlator。

这条链可以用下面的简化写法理解：

```

binned = bin_correlators(correlators, binsize=binsize)
indices = bootstrap_indices(n_blocks, draw_size, seed=seed, n_boot=n_boot)
resampled_means = bootstrap_means(binned, indices=indices)

```

这里的关键不是“把同一份数据重复算很多次”，而是“把可能相关的 configuration 先压成 block，再把 block 当成经验样本重新抽样”。这样得到的 bootstrap 分布，才更适合用来估计误差、协方差和后续拟合参数的不确定性。对于格点数据来说，这一步尤其重要，因为相邻 configuration 往往并不独立。

如果把这段逻辑写成更接近真实代码的顺序，就是 `apply_fold_t` 先利用时间反向对称性把 correlator 整理得更平滑，然后 `bin_correlators` 压低自相关，接着 `bootstrap_correlator_means` 生成重采样平均值，最后这些 bootstrap means 被送进 `effective_mass_with_bootstrap`、协方差估计和拟合模块。

所以 bootstrap、binning 和协方差分别负责三件不同的事：

- binning 减弱自相关；
- bootstrap 把样本波动传播到物理量；
- covariance 告诉我们不同时间点之间是不是一起涨、一起跌。

4.2.5 最小 χ^2 非线性拟合

当前 two-point 和 DA 拟合都采用最小 χ^2 的非线性最小二乘。对数据向量 d 和模型向量 $m(\theta)$ ，残差写成

$$r(\theta) = \frac{m(\theta) - d}{\sigma}, \quad (11)$$

或者在 correlated fit 时写成

$$r(\theta) = L^{-1}(m(\theta) - d), \quad (12)$$

其中 L 是协方差矩阵的 Cholesky 因子，满足 $C = LL^\top$ 。

目标函数是

$$\chi^2(\theta) = r(\theta)^\top r(\theta), \quad \chi^2/\text{dof} = \frac{\chi^2(\theta)}{N_{\text{data}} - N_{\text{param}}}. \quad (13)$$

如果把统计假设写完整一点，就是：在近似高斯误差下，最大化似然函数等价于最小化 χ^2 。所以这里的 fit 不是“凭感觉找一条最像的曲线”，而是一个有明确统计意义的参数估计问题。

代码使用 `scipy.optimize.least_squares(method="trf")` 来最小化这个目标。对学生来说，最重要的是理解：

- 小的 χ^2/dof 说明模型和窗口与数据较一致，但自动不等于“物理正确”；
- 过大的 χ^2/dof 往往表示窗口、模型或协方差处理有问题；
- correlated fit 更接近真实统计结构，但更依赖稳定协方差；
- bootstrap fit 让结果依赖整条重采样分布，而不是某一个单次最优点。

在代码里，`least_squares` 实际上最小化的是一串标准化残差。先用当前参数构造模型值 `m(theta)`，再与数据向量 `d` 相减得到 `delta = m(theta) - d`。如果是 uncorrelated fit，就把 `delta` 除以逐点误差 `sigma`；如果是 correlated fit，就先对 bootstrap 协方差做 Cholesky 分解，用 `L^{-1} delta` 作为残差。然后把这些残差平方求和，就是 `chi^2`。

为了让优化稳定，代码还会给参数做物理约束重参数化：

- amplitude 用 log 参数化，保证它们始终为正；
- energy 按 state 顺序写成正的基态能量和正的 gap，从而保证能级递增。

换成公式就是：

$$r_i(\theta) = \frac{m_i(\theta) - d_i}{\sigma_i} \quad \text{或} \quad r(\theta) = L^{-1}(m(\theta) - d), \quad (14)$$

$$\chi^2(\theta) = \sum_i r_i(\theta)^2 \quad \text{或} \quad \chi^2(\theta) = r(\theta)^\top r(\theta). \quad (15)$$

这也是为什么它叫 chi-square fit：它不是在直接最小化“模型和数据的差”，而是在最小化“按照统计不确定度标准化后的差”。只有当偏差相对于误差足够小，这个偏差才会被认为可以接受。

对学生来说，读 chi-square fit 结果时至少要同时看四件事：

1. 参数值是不是落在物理上合理的区间；
2. χ^2/dof 是否说明窗口和模型能接受这批数据；
3. 协方差是否稳定，是否需要 shrinkage；
4. bootstrap 后的参数分布是否和 mean fit 一致。

更具体地说，学生读拟合结果时应该同时问四个问题：

1. 这个模型是否真的符合该时间窗里的物理假设？
2. 当前窗口是否已经尽量避开 excited-state contamination？
3. 协方差矩阵是否稳定到足以支持 correlated fit？
4. 当前的 χ^2/dof 是“模型足够好”，还是“窗口太短、自由度太少”造成的表面好看？

也就是说， χ^2 不是一个孤立的数字，而是和窗口选择、模型选择以及协方差处理绑在一起看的。

4.2.6 bootstrap-centered 的中心值和误差

当前实现并不是只做一次拟合就结束，而是采用 bootstrap-centered 的汇总方式：

1. 先对 bootstrap mean 做 mean fit，得到 warm start；
2. 再对每个 bootstrap sample 做拟合；
3. 对所有成功样本的参数进行稳健汇总；
4. 用分位数而不是单次 Hessian 误差定义最终中心值和误差。

具体来说，当前代码用 16% 和 84% 分位数定义

$$\mu = \frac{P_{16} + P_{84}}{2}, \quad \sigma = \frac{P_{84} - P_{16}}{2}. \quad (16)$$

所以很多表格里的 central value / uncertainty，都是 bootstrap 分布的分位数摘要。

4.2.7 从 0-effective-mass 到 1-c2pt-fit

这一段是 Week 1 里最重要的过渡：学生不是在学两个孤立步骤，而是在学同一条物理链条里的前后两环。

前一节 0-effective-mass 做的事情很简单：从 correlator 直接看出 plateau 在哪里、噪声什么时候开始压过信号、bootstrap 误差条是否和样本波动一致。后一节 1-c2pt-fit 做的事情更进一步：在你已经看清楚 plateau 的前提下，用一个显式模型去提取 ground state energy 和 overlap。

这两步之间的逻辑关系是：

1. 0-effective-mass 先告诉你“哪一段时间窗看起来像单态区”；
2. 1-c2pt-fit 再把这段时间窗写成具体的拟合区间；
3. 如果 0-effective-mass 根本没有 plateau，1-c2pt-fit 里的任何漂亮数字都不可信；
4. 如果 0-effective-mass 的后端误差已经炸开，那么 1-c2pt-fit 不应该把那一段硬塞进窗口里。

因此，0-effective-mass 不是“前菜”，而是 1-c2pt-fit 的诊断图。它告诉学生：你接下来应该在哪些时间点上认真建模，哪些时间点只适合用来说明“不该拟合到这里”。

推荐的阅读和动手顺序

1. 先回看 0-effective-mass 的图，标出你认为最稳定的 plateau 区域；
2. 再打开 1-c2pt-fit 的 notebook，看它是如何把这个窗口变成 $t_{\min}/t_{\max}$ ；
3. 对比不同 $t_{\min}$ 下的 χ^2/dof 、能量和 overlap；
4. 把 notebook 里试出来的稳定窗口写进 input file；
5. 重新跑一遍，检查 notebook 和 input file 是否给出一致结论。

带着问题去练习 学生在从 0-effective-mass 过渡到 1-c2pt-fit 时，至少要能回答下面这些问题：

1. 为什么 plateau 区域比前端更适合做 fit？
2. 为什么后端即使“看起来更平”，也不一定更可信？
3. 如果 plateau 只有很短一段，应该优先缩窄窗口还是增加 state 数？
4. 为什么 k_0 和 k_6 的最佳窗口可能不同？
5. 为什么 notebook 中试出来的窗口必须被 input file 冻结下来，才能算真正复现？

这一段的练习题

1. 任选一张 0-effective-mass 图，圈出你认为最适合进入 1-c2pt-fit 的候选时间窗，并说明理由。
2. 任选另一张图，故意把窗口往前移动一个时间点，再比较 χ^2/dof 和能量是否变差。
3. 用一句话说明：为什么 effective mass 只能告诉你“哪里像 plateau”，却不能替代真正的 correlator fit?
4. 写下你最终选定的 $t_{\min}$ ，并说明这个选择是从图像直觉、统计稳定性还是物理可解释性出发的。

这组练习的目的不是把学生训练成“会报一个数字的人”，而是训练他们先看图、再建模、最后才接受拟合结果。

4.2.8 0-effective-mass: 把概念变成图

在真正进入两点拟合之前，学生应该先把 `examples/l64c64a076_m140/analysis/0-effective-mass/` 跑通。这个阶段只做一件事：从两点 correlator 直接构造 effective mass，并把结果画出来。它不是最终物理提取，而是把前面学到的统计概念落到一张图上。

运行方式 这个最小练习有两种等价的跑法：

- **notebook 方式**: 打开 `examples/l64c64a076_m140/analysis/0-effective-mass/meff_k0.ipynb` 和 `meff_k6.ipynb`，逐格执行，先看输入数据、再看 effective mass 公式、最后看图。
- **input file 方式**: 打开 `examples/l64c64a076_m140/input_files/0-effective-mass/meff_k0.txt` 和 `meff_k6.txt`，用同一套底层 CLI 跑一遍，把 notebook 里调出来的参数冻结成可复现版本。

两种方式的分工很明确：notebook 负责理解，input file 负责复现。前者允许学生临时检查中间量，后者要求参数固定、输出稳定。

预期结果 跑完之后，学生应该看到三类结果：

- 一张或两张 effective mass 图，图上能看出前端弯曲、中间 plateau、后端噪声放大这三种行为；
- bootstrap 误差条已经正确反映样本波动，而不是只剩一条平线或完全失控；
- k_0 和 k_6 的图在 plateau 起点、噪声增长速度或误差大小上可能不同，这正是后续选窗口要利用的信息。

如果跑完以后图像没有 plateau、误差条明显不合理，或者 notebook 和 input file 的结果对不上，那就说明这一步还没有学透，不应该急着进入 1-c2pt-fit。

这一步最适合训练学生的地方，是让他们在图像里看到三个统计事实：

- 小 t 处 excited-state contamination 往往更强；
- 中间可能出现一段近似平坦的 plateau；
- 大 t 处误差条快速变大，读到的往往主要是噪声。

Week 1 的训练目标有四个，而且每一个都要对应一次马上动手的练习：

1. 识别 correlator 在不同 t 区间的噪声增长；
2. 通过 effective mass 观察 plateau 是否开始形成；
3. 理解 bootstrap 平均值、误差条和样本波动之间的关系；
4. 学会从图上判断哪些时间点适合以后进入 1-c2pt-fit 的拟合窗口。

这个练习的核心不是“算一个漂亮的数”，而是让学生先回答下面这些问题：

- effective mass 在小 t 和大 t 的行为为什么不同；
- 哪些点看起来已经接近 plateau；
- 误差条什么时候开始快速变大；
- k_0 和 k_6 的有效质量图在噪声结构上有什么不同；
- 为什么 effective mass 只能作为窗口选择的诊断，而不是最终物理结果。

对学生来说，0-effective-mass 的正确用法是：

1. 先跑 notebook 版，目的是看中间量、熟悉图形、观察 plateau；
2. 再跑 input file 版，确认同样的数据、同样的参数会得到同样的 effective-mass 结果；
3. 把这一步的观察结果记下来，作为后面 1-c2pt-fit 选择拟合窗口的依据。

如果学生连 effective mass 图都看不懂，就不要急着进入两点拟合，更不要直接跳到 DA ratio。

4.2.9 如何读 effective mass 图

一张 effective mass 图通常要同时看三件事：均值、误差条和时间依赖趋势。学生第一次看图时，不要急着问“这个数是不是对”，先问下面几个更基本的问题：

1. 图的前端是不是还在明显弯曲？如果是，说明 excited-state contamination 还比较强；
2. 中间有没有一段比较平的区域？如果有，这往往就是后面 1-c2pt-fit 的候选窗口；
3. 图的后端误差是不是迅速放大？如果是，说明继续往后看已经主要是在读噪声；
4. 不同 momentum 的图是不是有不同的 plateau 起点？如果有，说明拟合窗口不能机械统一；
5. bootstrap 误差是不是和肉眼看到的波动一致？如果不一致，要先怀疑统计处理，而不是急着解释物理。

effective mass 图最重要的用途不是“报一个最终值”，而是帮助学生学会区分三种区域：

- **前端**：信号仍然受激发态污染，数值变化快，但物理解释不稳；
- **平台区**：局部斜率近似稳定，是后续拟合最有价值的区域；
- **后端**：噪声快速上升，图像看似平，但那种“平”往往只是统计涨落主导。

学生在 Week 1 中要练的，就是把这三类区域分辨出来。

4.2.10 Week 1 的练习与检查

为了确保学生不是“看过一遍就算学了”，Week 1 的前半段应该至少完成下面的练习：

1. 分别画出 k_0 和 k_6 的 effective mass 图，并用一句话描述它们 plateau 的差别；
2. 标出每张图中你认为最可能进入 1-c2pt-fit 的时间窗；
3. 观察 bootstrap 误差条在不同 t 的变化，解释为什么后端更难拟合；
4. 对 notebook 和 input file 两种方式各跑一次，确认输出图和表格一致；
5. 写一段简短记录，说明你为什么认为某个窗口更适合拟合，而不是另一个窗口。

这些练习的目的不是让学生“猜对标准答案”，而是训练他们形成一致的判断逻辑：看到图 -> 找 plateau -> 想噪声 -> 再联系拟合窗口。

4.2.11 DA ratio 的最小物理图像

Month 1 的学生还需要知道后面的 DA 流程是如何接在两点拟合之后的。当前代码里的最小 ratio 结构是

$$R(t) = \frac{\mathcal{N}(t)}{C_{2pt}(t)}. \quad (17)$$

对于 T5 通道，分子有最小形式

$$\mathcal{N}_{T5}(t) \propto \sum_n \frac{1}{2} \sqrt{A_n 2E_n} m_n \left(e^{-E_n t} - e^{-E_n(N_t-t)} \right), \quad (18)$$

而 Z5 通道额外带有动量因子

$$\mathcal{N}_{Z5}(t) \propto \sum_n \frac{1}{2} \sqrt{A_n 2E_n} \frac{P_z}{E_n} m_n \left(e^{-E_n t} - e^{-E_n(N_t-t)} \right). \quad (19)$$

这就是为什么 Month 1 必须先把两点拟合学透：DA ratio 的模型参数直接依赖两点拟合得到的 A_n 和 E_n 。

这一步到底在做什么 1-c2pt-fit 不是在“把一条曲线硬塞给数据”，而是在一个已经被 0-effective-mass 诊断过的时间窗里，回答两个具体问题：

1. ground-state energy 是多少；
2. ground-state overlap 是多少。

换句话说，0-effective-mass 负责告诉你“哪里像单态区”，而 1-c2pt-fit 负责把“像单态区”这件事变成可复现的参数提取。前者给直觉，后者给数值；前者看图，后者建模。

因此，学生在进入 1-c2pt-fit 之前，至少应该已经能从 effective mass 图上指出：

- 哪一段时间窗最像 plateau；
- 为什么更早的点不适合直接拿来拟合；
- 为什么更晚的点虽然噪声可能更大，但不一定更可信；
- 为什么不同 momentum 往往需要不同的窗口。

这一节的练习题

1. 先看 0-effective-mass 的图，给 k0 和 k6 各选一个你认为最稳的候选窗口，并写下理由。
2. 只允许你改一个参数：如果把 tmin 向前或向后移动一个时间点，你预期 χ^2/dof 会怎么变，为什么？
3. 比较 1-state 和 2-state 的任务差别：前者为什么更适合先做窗口扫描，后者为什么更依赖 prior？

4. 如果 0-effective-mass 显示 plateau 很短, 你会先考虑扩大窗口还是增加 state 数? 把你的判断逻辑写出来。

这组练习的目标, 是让学生带着问题进入拟合, 而不是先把拟合跑完再回头解释结果。

4.2.12 1-c2pt-fit: 从 plateau 到参数提取

1-c2pt-fit 是 Week 1 里真正把“看图”变成“提参”的那一步。它不是重复 0-effective-mass, 而是用一个明确的 correlator 模型, 把已经看起来接近 plateau 的那段区间转成 ground-state energy 和 overlap 的数值估计。

对两点函数来说, 最基本的物理图像是: 在有限时间延拓下, correlator 可以写成若干个态的叠加。最简的单态近似当然是

$$C_{2pt}(t) \approx A_0 e^{-E_0 t}, \quad (20)$$

但真实数据在早时刻往往还混有 excited states, 所以更现实的模型是

$$C_{2pt}(t) = \sum_{n=0}^{N-1} A_n K_n(t), \quad (21)$$

其中 kernel $K_n(t)$ 的具体形式取决于 correlator 类型。对当前 example 来说, 学生要先记住的不是每个符号的代码名字, 而是下面这件事: **更高 state 数并不是“越多越好”, 而是“是否足以解释你选定的时间窗”。**

因此, 1-c2pt-fit 的任务可以拆成三个层次:

1. 先选一个看起来已经接近单态区的窗口;
2. 再用 1-state / 2-state / 3-state 模型去解释这段窗口;
3. 最后比较不同模型下的参数稳定性、 χ^2/dof 和 bootstrap 波动。

这也是为什么 1-c2pt-fit 的学习顺序必须跟着 0-effective-mass 走: 前者不先看图, 后者就没有合理的窗口起点。

运行方式

- **notebook 方式:** 打开 `examples/l64c64a076_m140/analysis/1-c2pt-fit/` 里的 notebook, 先观察 effective mass, 再看拟合窗口、state 数和输出表格。
- **input file 方式:** 打开对应的 `examples/l64c64a076_m140/input_files/` 下的两点拟合输入文件, 把 notebook 里试出来的窗口和 prior 固化下来。

Notebook 的角色是“试”, input file 的角色是“定”。学生应该先在 notebook 里看懂不同窗口为什么会改变结果, 再把最终选择写进 input file, 确认结果能被重复得到。

预期结果 跑完 1-c2pt-fit 后，学生应该至少看到以下几类输出：

- 一张两点 correlator 的拟合图，图上能看到数据点、拟合曲线和候选窗口；
- 一个结果表，里面有 ground-state energy、overlap、 χ^2/dof 和 p-value；
- 不同 state 数或不同 tmin 下结果的对比；
- bootstrap 汇总后的中心值和不确定度。

如果图上没有明显的拟合区间，或者不同输入方式得到的结果不一致，那么这一步还没有真正学会，不应该急着进入 DA ratio。

这一节的练习题

1. 先从 0-effective-mass 的图里选出一个你认为最稳的窗口，再在 1-c2pt-fit 里验证这个窗口是否真的稳定。
2. 比较 1-state 和 2-state：如果 1-state 的 χ^2/dof 很高，而 2-state 明显下降，这说明了什么？
3. 故意把 tmin 改到更早一点，观察结果如何变化，并解释这通常意味着什么物理污染。
4. 把 notebook 里调过的参数写进 input file，再跑一遍；如果输出不同，先检查哪里没有冻结。

这一步应该形成的判断习惯 学生在 1-c2pt-fit 中要养成的习惯是：

- 先看 plateau 再看拟合值；
- 先看窗口再看 χ^2/dof ；
- 先比较不同 state 数，再决定是否加复杂度；
- 先把 notebook 理顺，再把结果冻结到 input file。

为什么窗口、state 数和 prior 必须一起看 这三件事不是三个互不相干的按钮，而是一组彼此约束的统计选择。

- **窗口** 决定你把哪些时间点当成“可被模型解释”的数据；
- **state 数** 决定你愿意用多复杂的物理模型去解释这段数据；
- **prior** 决定你在数据不够强时，愿意借用多少外部物理知识来稳定拟合。

这三者的关系可以理解成一个三角形：

- 窗口太短，拟合可能很稳，但参数会偏；

- 窗口太长，后端噪声会把拟合拖垮；
- state 数太少，模型解释不了 plateau 前后的弯曲；
- state 数太多，参数会变得不稳定，甚至把噪声拟合成物理；
- prior 太弱，拟合会漂；prior 太强，又可能把数据“压”回先验中心。

所以学生不要把这些参数看成独立可调项。正确的顺序是：先用 0-effective-mass 判断大概的窗口，再用 1-state 看地基是否稳，再考虑 2-state 或 3-state，最后才决定需不需要 prior 来固定或约束低态。

在本项目里，`low_state_prior_tmin[pz]` 的角色尤其重要：它不是“随便挑一行表”，而是告诉下一层拟合“上层里哪一个稳定结果最值得拿来当先验”。同理，`pz0_ground_energy` 也不是装饰参数，而是给 ground state 一个物理上合理的起点，避免优化器在明显不合理的区域里乱跑。

这一节的练习题

1. 如果你把窗口往前推一点， χ^2/dof 可能变小也可能变大。请用 excited-state contamination 的语言解释这两种情况各意味着什么。
2. 如果你把窗口往后推一点，误差条通常会变大。请解释为什么这并不自动意味着结果更好。
3. 比较 1-state 和 2-state：在什么情况下“多一个 state”是真正改善模型，在什么情况下只是让参数更多？
4. 为什么 prior 只能帮助稳定拟合，而不能替代窗口选择？请给出你的物理理由，而不是只写“因为代码这么做”。

4.2.13 两点拟合的窗口、状态数和先验选择

对学生来说，最重要的不是记住某一组固定数字，而是理解当前两点拟合的决策顺序。

推荐顺序

1. 先跑 1-state，只看 ground-state 是否稳定；
2. 在 1-state 的 plateau 上选一个稳定的 tmin；
3. 用这个稳定行作为 `low_state_prior_tmin[pz]` 的来源；
4. 再跑 2-state；
5. 只有当 2-state 不能解释数据时，才考虑 3-state。

窗口参数

- `tmin_window` 决定扫描哪些起点，主要用来避开早时刻的 excited-state contamination；
- `tmax` 决定晚时刻是否已经进入噪声主导区；
- 在实际使用中，`tmin` 通常比 `tmax` 更重要；
- 更高 state 数的窗口通常比低 state 数更窄，因为模型更容易过拟合。

先验和锚定 当前实现把 two-point 的关键物理约束拆成两类：

- `pz0_ground_energy`: 来自可信的 $p_z = 0$ 参考值，用来给 E_0 提供初始猜测；如果打开 `fix_ground_energy_from_dispersion`，它还会直接固定 E_0 ；
- `low_state_prior_tmin[pz]`: 从前一个 state 的 fit table 中挑选某一行，作为下一个 state 的 prior 来源。

这两个量的角色不同：

- `pz0_ground_energy` 是物理锚点，回答“ground state 大概应该在哪”；
- `low_state_prior_tmin` 是表格行选择器，回答“上一层拟合里哪一行最适合作为下一层的先验”。

状态数如何递增

- 1-state: 没有低态先验，只做最基础的 ground-state 提取；
- 2-state: 通常对 E_0 加 prior 或固定锚定，并让 E_1 吸收 excited-state correction；
- 3-state: 在 2-state 的基础上再多一个 excited level，先验的作用更强，窗口也应该更保守。

协方差与 shrinkage 当前代码优先尝试 correlated fit。若协方差矩阵不稳定，代码会按照

$$\lambda \in \{0.0, 0.1, 0.2, 0.3, 0.5, 1.0\} \quad (22)$$

的顺序尝试 shrinkage。这里：

- $\lambda = 0$ 表示完全保留协方差；
- $\lambda = 1$ 表示只保留对角项；
- 中间值是在稳定性和相关性信息之间做折中。

学生应该理解：correlated fit 更接近真实统计结构，但也更依赖稳定协方差；如果协方差不稳，先用 shrinkage 稳住数值，再讨论物理解释。

4.2.14 DA ratio 的窗口和诊断逻辑

DA ratio 的拟合不是“随便拿一段时间窗做一个比值”，而是一个和 two-point 拟合强耦合的流程。

当前输入约束

- `fit_target = ratio;`
- `fit_component` 可以是 `real`、`imag` 或 `both`;
- 真实 example 只保留 `b_X`、`OT5`、`bT = 0`;
- `decay_constant_check = true` 时，代码会强制只看 `real part`，并把 `bTlist` 与 `bzlist` 收缩到 0;
- `two_point_fit_root` 和 `two_point_fit_window_by_pz` 决定 DA ratio 使用哪一行 two-point 参考。

窗口选择 DA 的 `fit_window` 是核心调节钮。正常模式下，学生应该直接指定每个 momentum 的基准窗口；`decay_constant_check` 模式下，代码会围绕基准窗口做一个小范围扫描，用来检查：

- 抽出的 `decay constant` 是否对 `tmin` 稳定；
- `tmax` 是否已经进入噪声主导区；
- 结果是否明显受 `excited-state contamination` 影响；
- 不同窗口之间的 χ^2/dof 和相对误差是否相互一致。

两点与 DA 的连接 DA ratio 不是独立于 two-point 的。当前代码里，two-point 的拟合结果直接进入 DA ratio 的模型：

- two-point 的 A_n 和 E_n 决定 DA 分子和分母的时间依赖；
- `T5` 和 `Z5` 只是不同的算符通道，底层都依赖同一套 two-point 约束；
- `two_point_fit_sample_coupled` 若打开，则每个 bootstrap 样本会尽量使用对应的 two-point sample 行，而不是只看 summary table。

所以 Week 1 的思路应该是：先把两点拟合窗口选稳，再让 DA ratio 在这个稳态上做提取，而不是反过来。

4.2.15 Week 1 和后续 example 的连接

当前仓库中的实际分析顺序是：

阶段	example 目录	核心任务
0-effective-mass	examples/.../analysis/0-effective-mass/	先只算 effective mass 和画图，练习 plateau、bootstrap 和噪声诊断
1-c2pt-fit	examples/.../analysis/1-c2pt-fit/	提取两点能量和 overlap，建立窗口选择规则
2-bm	examples/.../analysis/2-bm/	提取 DA bare matrix element
3-normalize	examples/.../analysis/3-normalize/	归一化，当前只保留 mode3
4-FT	examples/.../analysis/4-FT/	做 Fourier 变换，检查最终分布

Week 1 只要求学生真正吃透第一阶段，但应该知道后面三步为什么存在、每一步为什么都依赖前一步。

4.3 如何运行当前 example：先看原则，细节放到后面的实操部分

本仓库当前保留了两种等价但用途不同的运行方式：notebook 和 input file。

这里先给学生一个总原则：

- 先 notebook，后 input file；
- notebook 用来理解流程、检查中间量、调窗口；
- input file 用来冻结参数、复现结果、提交最终版本；
- 两者必须走同一条底层分析链，否则结果不应被认为是同一个 example。

具体到当前 example，真实的操作顺序和命令细节放在下面的实操任务里。

4.4 Week 1 的实操主线：环境、代码理解与 golden example 复现

4.4.1 Week 1 目标

第一周的目标是复现已有分析流程，并把流程看懂、记录清楚。这周不要急着重构，不要急着做 agent。先要回答三个问题：

1. 这个 meson DA 分析样例从哪些输入开始？
2. 中间经过哪些步骤？
3. 最终产生哪些物理结果、图和检查量？

4.4.2 Week 1 的学习-练习循环

Week 1 不应该被理解成“先把工具书看完，再统一上手”。更合理的做法是：每学一个分析模块，就立刻把它放进 notebook 或 input file 里验证一次，再把验证结果写回文档。

学生需要边学边练的内容包括：

- Git 基本操作；
- Python 虚拟环境或 conda 环境；
- 项目目录结构；
- 如何运行现有 notebook 或 script，并立刻核对输出是否符合预期；
- meson DA 分析中的基本对象：2pt、3pt/ratio、bare matrix element、 z dependence、 P_z dependence、real/imag symmetry；
- 什么是 golden example，为什么它可以作为 regression case；
- 如何把 notebook 里摸索出来的参数冻结到 input file 里。

4.4.3 Week 1 实操主线：先 notebook，再 input file

Week 1 的实操不是“随便点一个 notebook 看看”，而是按固定顺序把同一条分析链跑两遍。

第一遍：Notebook 探索 第一遍的目标是看懂流程和中间量，不追求最后一步就完全冻结成最终版。

1. 先打开 `examples/l64c64a076_m140/analysis/0-effective-mass/` 里的 notebook；
2. 先看 effective mass 的 bootstrap 结果和图，熟悉 plateau、噪声和误差条；
3. 再打开 `examples/l64c64a076_m140/analysis/1-c2pt-fit/` 里的 notebook；
4. 从 1-c2pt-fit 开始看两点 correlator 的 plateau 和拟合窗口；
5. 选出稳定的 tmin，确认 χ^2/dof 、能量和 overlap 没有明显异常；
6. 再打开 `examples/l64c64a076_m140/analysis/2-bm/`，用同一套 two-point 参考去看 DA bare matrix element；
7. 接着打开 `examples/l64c64a076_m140/analysis/3-normalize/`，只保留 mode3 路径；
8. 最后打开 `examples/l64c64a076_m140/analysis/4-FT/`，检查最终 Fourier 结果的形状和物理趋势。

Notebook 这一遍的重点是：

- 理解每一步输出什么；
- 找到每一步里最敏感的参数；
- 把 notebook 里临时调过的窗口记下来；
- 不要急着改代码结构。

第二遍：Input file 冻结与复现 第二遍的目标是把 notebook 里已经理解清楚的参数，写进 input file，然后用 CLI 重跑，确认结果一致。

当前 example 里常见的 input file 是：

- `examples/l64c64a076_m140/input_files/0-effective-mass/meff_k0.txt`;
- `examples/l64c64a076_m140/input_files/0-effective-mass/meff_k6.txt`;
- `examples/l64c64a076_m140/input_files/2-bm/da_nstate_k0.txt`;
- `examples/l64c64a076_m140/input_files/2-bm/da_nstate_k6.txt`;
- `examples/l64c64a076_m140/input_files/3-normalize/da_normalize_mode3.txt`;
- `examples/l64c64a076_m140/input_files/4-FT/da_fourier_mode3.txt`。

Input file 的运行要求是：

1. 参数必须和 notebook 一致，尤其是数据路径、窗口、state 数、模式和输出目录；
2. 运行顺序必须和 notebook 的分析链一致，先两点，再 bare matrix element，再 normalize，最后 Fourier；
3. 跑完以后要检查输出文件名、表格和图是否与 notebook 对齐；
4. 如果不一致，先检查 input file 是否漏了参数，再检查 notebook 是否靠了手动状态。

对学生最重要的判断标准是：**notebook 和 input file 应该得到同一物理结论**。如果二者不一致，就说明参数冻结还不彻底，不能把结果当作 final version。

4.4.4 Week 1 每日任务单

Week 1 的后半段采用“先看当天目标，再完成当天练习，再交当天产出”的方式推进。每一天都尽量只抓一件主事：先把当天该学的内容吃透，再把它变成一个可检查、可复现的小结果。

天数	当天主题	当天主要产出	当天检查点
Day 1	环境准备与 repo 初读	<code>docs/repo_initial_notes.m</code>	知道 repo 结构和 golden example 在哪里

Day 2	手动复现 golden example	docs/golden_example_reproduction.md	能从干净环境复现一次 与运行 log 主流程
Day 3	画 data-flow map	docs/data_flow_map.md	能把流程拆成输入、输出 和物理含义
Day 4	识别 config 参数	docs/config_design_notes.draft	能指出哪些值必须 和 draft YAML fig 化
Day 5	总结与 review	docs/week1_report.md	能说明自己做了什么、还 卡在哪里

Day 1: 环境准备与 repo 初读 课堂动作:

- 1. clone 或进入项目 repo;
- 2. 创建 Python 环境;
- 3. 安装依赖;
- 4. 运行现有测试;
- 5. 阅读 README 和 examples 目录;
- 6. 写下当前目录结构说明。

建议命令:

```
git status
python --version
pip list
pytest
find . -maxdepth 3 -type f | sort | head -100
```

要交的东西:

```
docs/repo_initial_notes.md
```

内容包括:

- 当前 repo 里有哪些主要目录;
- 哪些目录是数据、代码、配置、输出、文档;
- 当前是否能成功运行测试;
- 当前不理解的问题列表。

Codex prompt 示例:

```
We are starting an AI-assisted LQCD meson DA analysis-agent project.  
Please inspect the repository structure and write a short developer-oriented summary  
in `docs/repo_initial_notes.md`.
```

Focus on:

- main code directories;
- example analysis directories;
- likely inputs and outputs;
- existing scripts or notebooks;
- tests if any.

Do not modify source code. Only create or update the documentation file.

验收:

- 学生能说清楚 repo 大致结构;
- 知道 golden example 在哪里;
- 知道如何运行最基本命令;
- 有第一份文档记录。

Day 2: 手动复现 golden example 课堂动作:

1. 按现有方式运行 golden example;
2. 记录每一条命令;
3. 记录需要哪些输入文件;
4. 记录生成哪些输出文件;
5. 保存运行 log。

要交的东西:

```
docs/golden_example_reproduction.md  
outputs/golden_example_manual/run.log
```

文档中必须包含:

- 运行日期;
- git commit hash;
- Python 环境;
- 输入文件路径;

- 运行命令；
- 输出文件列表；
- 运行中遇到的问题。

建议记录 git commit:

```
git rev-parse HEAD
```

Codex prompt 示例:

```
I manually ran the golden example. Below are the commands and output files.
Please help me turn my notes into a clean reproduction document at `docs/
golden_example_reproduction.md`.
```

Requirements:

- include environment information;
- include exact commands;
- include input files;
- include output files;
- include unresolved questions;
- do not modify code.

Here are my raw notes:

[paste notes]

验收:

- 能从干净环境复现一次 golden example;
- 所有命令被记录;
- 输出文件被列出;
- 失败或 warning 没有被隐藏。

Day 3: 画 data-flow map 课堂动作:

把 golden example 拆成流程图或表格, 例如:

```
raw data
-> data loading
-> 2pt fit input
-> ratio construction
-> ratio fit
-> bare matrix element
-> plots and summary
```

要交的东西:

docs/data_flow_map.md

建议表格：

步骤	输入	输出	物理/技术含义
Data audit	raw correlator files	file checklist	检查数据是否齐全
2pt fit	two-point correlator	mass/energy fit result	提供能量和 overlap 信息
Ratio analysis	3pt/2pt ratio	bare matrix element	提取矩阵元
Plotting	summary table	figures	检查稳定性和物理趋势

Codex prompt 示例：

```
Please create `docs/data_flow_map.md` for the golden example.
```

```
Use the current analysis notes and file list below.
```

```
The document should explain:
```

- each analysis step;
- input files for each step;
- output files for each step;
- what physical or technical quantity each step produces;
- which steps should later become separate pipeline stages.

```
Do not modify source code.
```

```
Notes:
```

```
[paste notes]
```

验收：

- 学生能画出完整分析流程；
- 知道哪些步骤可以 later 变成 CLI module；
- 知道哪些输出应该进入 summary.csv/result.json；
- 知道哪些检查属于 physics sanity check。

Day 4：识别 config 参数和硬编码路径 课堂动作：

检查现有脚本或 notebook 中哪些值应该变成 config。

典型 config 字段包括：

- ensemble name；

- input data root;
- output directory;
- momentum list;
- z range;
- bootstrap/jackknife 参数;
- fit model;
- fit window;
- plot options;
- random seed;
- overwrite policy。

要交的东西:

```
docs/config_design_notes.md
configs/golden_example_draft.yaml
```

Codex prompt 示例:

```
Inspect the golden example scripts/notebooks and identify values that should become
configuration fields.

Please create:
1. `docs/config_design_notes.md`
2. `configs/golden_example_draft.yaml`

The config should include paths, ensemble metadata, momentum/z selections, fit
settings, bootstrap settings, output settings, and overwrite policy.

Do not refactor the analysis code yet.
Do not change existing outputs.
```

验收:

- 学生能指出哪些地方硬编码了路径或参数;
- 第一版 YAML config 能表达 golden example;
- 学生理解 config-driven pipeline 的意义。

Week 1 命令示例

```
git status
python --version
pip list
pytest
git rev-parse HEAD # 记录 commit hash
```

Week 1 常见错误

- **只跑 notebook，不记录命令**：必须记录每一条命令和对应的输出。
- **一开始就让 Codex 写完整系统**：每天只做一个小任务，不要一次要求 Codex 完成整个 Month 1。
- **不理解 effective mass 就继续**：如果看不出来 plateau，先回看 Week 1 的物理背景部分。

Week 1 导师检查问题

1. 学生是否真的跑通了一遍 golden example?
2. 是否记录了完整命令和 git commit hash?
3. 是否理解了主要输入输出?
4. 是否知道哪些参数应该 config 化?
5. 是否有不理解的问题被明确列出?

Day 5: Week 1 总结与导师 review 课堂动作:

整理第一周报告。

要交的东西:

```
docs/week1_report.md
```

报告模板:

```
# Week 1 Report

## 1. Goal

## 2. What I ran

## 3. Inputs and outputs

## 4. Data-flow map

## 5. Config parameters identified
```



```
## 6. Problems encountered

## 7. What I learned

## 8. Questions for advisor

## 9. Plan for Week 2
```

导师 review 重点：

1. 学生是否真的跑通过 golden example?
2. 是否记录了完整命令?
3. 是否理解主要输入输出?
4. 是否知道哪些参数应该 config 化?
5. 是否有不理解的问题被明确列出?

5 Week 2: Config-driven CLI 与 Pydantic schema

5.1 Week 2 目标

第二周的目标是把 golden example 从“手动运行”推进到“config-driven CLI”。

学生要理解：科研分析不能长期依赖 notebook 里手动改路径、手动运行 cell、手动保存图。真正可复现的分析应该由 config 驱动，并且每次运行都留下结构化结果和日志。

5.2 Week 2 学习内容

学生需要学习：

- Python package 基础结构；
- CLI 入口；
- YAML/JSON config；
- Pydantic schema；
- result.json 和 summary.csv 的区别；
- run manifest 的作用。

5.3 Week 2 任务 1: 定义 config schema

创建:

```
src/lqcd_agent/schema/analysis_config.py
```

建议 schema 包含:

```
from pydantic import BaseModel, Field
from typing import List, Optional

class EnsembleConfig(BaseModel):
    name: str
    lattice_size: Optional[str] = None
    pion_mass: Optional[float] = None
    notes: Optional[str] = None

class IOConfig(BaseModel):
    input_root: str
    output_dir: str
    allow_overwrite: bool = False

class AnalysisSelection(BaseModel):
    momenta: List[str] = Field(default_factory=list)
    z_values: List[int] = Field(default_factory=list)

class FitConfig(BaseModel):
    model: str
    tmin: Optional[int] = None
    tmax: Optional[int] = None
    bootstrap_samples: int = 500

class GoldenExampleConfig(BaseModel):
    run_id: str
    ensemble: EnsembleConfig
    io: IOConfig
    selection: AnalysisSelection
    fit: FitConfig
```

这只是示意，学生应根据真实分析流程修改字段。

Codex prompt 示例:

```
We are converting the golden example into a config-driven CLI.
```

```
Please implement Pydantic config schemas in:
```

```
`src/lqcd_agent/schema/analysis_config.py`
```

```
Use the draft YAML config in `configs/golden_example_draft.yaml` as the reference.
```

Requirements:

- define nested schemas for ensemble metadata, IO paths, analysis selections, fit settings, and runtime options;
- validate that output_dir is not empty;
- validate that bootstrap_samples is positive;
- validate that z_values are integers;
- add a helper function `load\_config(path: str) -> GoldenExampleConfig`;
- add pytest tests for valid config and invalid config.

Do not change the analysis algorithm yet.

验收:

- YAML config 可以被 load;
- 错误 config 会清楚报错;
- pytest 覆盖基本 validation;
- 学生能解释每个 config 字段的作用。

5.4 Week 2 任务 2: 定义 result schema

创建:

```
src/lqcd_agent/schema/analysis_result.py
```

建议 result schema 包含:

```
from pydantic import BaseModel
from typing import Dict, List, Optional

class OutputFiles(BaseModel):
    manifest: str
    result_json: str
    summary_csv: str
    plots: List[str]
    run_log: str

class FitSummary(BaseModel):
    status: str
    model: Optional[str] = None
    chi2_dof: Optional[float] = None
    notes: List[str] = []

class AnalysisResult(BaseModel):
    run_id: str
```

```

status: str
git_commit: Optional[str]
config_path: str
outputs: OutputFiles
fit_summary: Optional[FitSummary]
warnings: List[str] = []
errors: List[str] = []

```

Codex prompt 示例:

```

Please implement result schemas for the golden example run in:
`src/lqcd_agent/schema/analysis_result.py`

Requirements:
- include run_id, status, git_commit, config_path;
- include output file paths: manifest, result_json, summary_csv, plots, run_log;
- include fit summary fields if available;
- include warnings and errors;
- add tests that validate a successful result and reject a malformed result.

Do not run the analysis in this task.

```

5.5 Week 2 任务 3: 实现最小 CLI

创建:

```
src/lqcd_agent/cli/run_golden.py
```

CLI 的第一版可以只是:

1. 读取 config;
2. 创建 output directory;
3. 写 manifest.json;
4. 调用已有分析脚本或占位函数;
5. 收集输出文件;
6. 写 result.json 和 run.log。

建议命令:

```
python -m lqcd_agent.cli.run_golden --config configs/golden_example.yaml
```

Codex prompt 示例:

Please implement a minimal CLI for running the golden example:

```
`src/lqcd_agent/cli/run_golden.py`
```

Requirements:

- command: ``python -m lqcd_agent.cli.run_golden --config configs/golden_example.yaml``
- load and validate the Pydantic config;
- create the output directory;
- refuse to overwrite existing output unless `allow_overwrite` is true;
- write ``manifest.json`` with `run_id`, `timestamp`, `git commit`, `config path`, and environment summary;
- write ``run.log``;
- for now, call a placeholder function or existing script hook for the actual analysis ;
- write ``result.json`` using the result schema;
- add a CLI smoke test using a temporary directory.

Do not modify raw data.

Do not implement Snakemake yet.

验收标准:

- CLI 能从 config 启动;
- 不允许默认覆盖输出;
- `manifest.json` 存在;
- `result.json` 符合 schema;
- CLI smoke test 通过。

5.6 Week 2 命令示例

```
python -m lqcd_agent.cli.run_golden --config configs/golden_example.yaml  
pytest tests/test_schema.py tests/test_cli_smoke.py -q
```

5.7 Week 2 常见错误

- **config 字段太随意**: 所有影响结果的参数必须进 config, 用 Pydantic Field 加约束。
- **没有 schema 直接读 dict**: 后续 agent 无法可靠读取 config/result。
- **CLI 没有 overwrite protection**: 默认不应覆盖已有输出。

5.8 Week 2 导师检查问题

- config 是否完整？所有影响结果的参数是否都进入了 config？
- Pydantic validation 是否会对错误 config 清晰报错？
- CLI 是否能在 clean environment 运行？
- 是否有 overwrite protection？

5.9 Week 2 任务 4：记录 Codex 使用日志

学生需要维护：

```
docs/codex_usage_log.md
```

记录格式：

```
## Entry 001
Date:
Goal:
Prompt summary:
Files changed:
Tests added:
Did it work? yes/no
What I checked manually:
Problems:
Follow-up prompt:
```

目的不是监控学生，而是训练学生学会评估 AI-assisted coding 的质量。

6 Week 3：数据检查、最小测试体系与输出规范

6.1 Week 3 目标

第三周的目标是让项目从“能运行”变成“更可靠”。具体来说，要加入数据检查、pytest 测试、输出规范和基本 numerical regression 思想。

6.2 Week 3 学习内容

学生需要学习：

- data audit 的意义；
- pytest 的基本写法；
- smoke test、unit test、regression test 的区别；
- manifest 和 checksum；
- 如何检查输出是否可复现。

6.3 Week 3 任务 1: data audit 模块 (Data Integrity Check / Correlator Analyst)

创建:

```
src/lqcd_agent/audit/data_audit.py
```

data audit 模块的职责:

1. 检查 input root 是否存在;
2. 检查 config 中指定的必要文件是否存在;
3. 检查输出目录是否安全;
4. 统计文件数量和基本 metadata;
5. 写 audit_report.json;
6. 不读取或修改大规模 raw data 内容。

Codex prompt 示例:

```
Please implement a data audit module (Data Integrity Check) for the golden example.
```

```
File:
```

```
`src/lqcd_agent/audit/data_audit.py`
```

```
Requirements:
```

- input: validated GoldenExampleConfig;
- check that input_root exists;
- check that required files or directories listed in config exist;
- collect file count and basic metadata if cheap;
- do not modify raw data;
- write `audit\_report.json` into output_dir;
- return a structured audit result object;
- add pytest tests using temporary directories.

```
The auditor should fail clearly if required inputs are missing.
```

验收标准:

- 缺文件时能清楚报错;
- audit_report.json 能生成;
- 测试覆盖正常和失败情况;
- 学生理解为什么 data audit 是 Analysis Engineer 之前的第一步。

6.4 Week 3 任务 2: 最小 pytest 体系

至少需要以下测试:

```
tests/test_schema.py
tests/test_data_audit.py
tests/test_cli_smoke.py
```

测试内容:

- valid config 可以通过;
- invalid config 会失败;
- data audit 检测缺失输入;
- CLI 可以在 temporary output directory 运行;
- result.json 可以被 result schema 重新读取。

学生需要运行:

```
pytest -q
```

Codex prompt 示例:

```
Please add a minimal pytest suite for the Month 1 golden example pipeline.

Tests required:
1. valid config loads successfully;
2. invalid config with negative bootstrap_samples fails;
3. data audit module fails when input_root is missing;
4. CLI smoke test writes manifest.json and result.json into a temporary output
   directory;
5. result.json can be validated by AnalysisResult schema.

Use tmp_path for tests.
Do not require real large raw data in unit tests.
Do not make tests depend on machine-specific paths.
```

6.5 Week 3 任务 2.5: 生成 correlator_diagnostics.json

基于 effective mass 和 bootstrap 结果, 生成 Correlator Analyst 的第一版输出:

```
outputs/<case_id>/correlator_diagnostics.json
outputs/<case_id>/effective_mass_summary.csv
outputs/<case_id>/effective_mass.pdf
```

correlator_diagnostics.json 至少包含: 每个 momentum 的 plateau 检测结果、bootstrap 误差评估、标记为 suspicious 的 correlator (如果存在)。

6.6 Week 3 任务 3: 输出规范

规定 golden example 每次运行必须输出：

```
manifest.json
result.json
summary.csv
run.log
audit_report.json
plots/
```

6.6.1 manifest.json 应包含

- run_id;
- timestamp;
- git commit;
- config path;
- output directory;
- Python version;
- key package versions;
- hostname 可选;
- random seed 可选。

6.6.2 result.json 应包含

- run status;
- key outputs;
- major numerical results;
- warnings;
- errors;
- whether human review is needed later。

6.6.3 summary.csv 应包含

对于 Month 1, 可以先简单记录输出文件和关键数值; 后续 Month 2 再扩展为 fit candidate table。

6.7 Week 3 命令示例

```
# data audit
python -m lqcd_agent.audit.run --config configs/l64c64a076_m140.yaml
# correlator diagnostics
python -m lqcd_agent.analysis.correlator_diagnostics \
    --config configs/l64c64a076_m140.yaml
# run all tests
pytest tests/ -q
```

6.8 Week 3 常见错误

- 用真实数据路径写测试：测试会不可迁移；用 `tmp_path` 创建临时目录。
- `data audit` 不检查 `output dir`：可能意外覆盖已有结果。
- `correlator diagnostics` 只画图不输出结构化 JSON：图是给人看的，JSON 是给后续 agent 读的。

6.9 Week 3 导师检查问题

- `data audit` 是否能检测缺失输入？
- `correlator diagnostics` 是否能标记 bad correlator？
- `pytest` 是否覆盖正常和失败情况？
- 学生能否解释为什么 raw data 不能被修改？

6.10 Week 3 验收标准

第三周结束时：

- CLI 已经不是黑盒运行，而是有 `manifest/audit/result/log`；
- `pytest` 至少有 5 个有效测试；
- 学生能解释 smoke test 和 unit test 的区别；
- 学生能说明为什么 raw data 不能被 Analysis Engineer 修改；
- 输出目录结构稳定。

7 Week 4: Month 1 集成、最小 Analysis Engineer 雏形与展示

7.1 Week 4 目标

第四周的目标是完成第一个月的集成展示：学生应该能用一个命令运行 golden example，并生成完整的结构化输出和复现报告。同时，开始引入 Analysis Engineer Agent 的雏形，但不要求完成完整多 agent 系统。

7.2 Week 4 学习内容

学生需要学习：

- Analysis Engineer Agent 的边界；
- task input 和 task output；
- 安全规则；
- 如何写 demo report；
- 如何向导师展示一个科研软件原型。

7.3 Week 4 任务 1：定义 Analysis Engineer task schema

创建：

```
src/lqcd_agent/schema/analysis_engineer_task.py
```

示意：

```
from pydantic import BaseModel
from typing import Optional, List

class AnalysisEngineerTask(BaseModel):
    task_id: str
    task_type: str
    config_path: str
    output_dir: str
    allow_overwrite: bool = False
    notes: Optional[str] = None

class AnalysisEngineerTaskResult(BaseModel):
    task_id: str
    status: str
    result_json: Optional[str] = None
    summary_csv: Optional[str] = None
    plots: List[str] = []
    warnings: List[str] = []
    errors: List[str] = []
```

Codex prompt 示例:

```
Please add a minimal Analysis Engineer task schema.

File:
`src/lqcd_agent/schema/analysis_engineer_task.py`

Requirements:
- define AnalysisEngineerTask with task_id, task_type, config_path, output_dir,
  allow_overwrite, notes;
- define AnalysisEngineerTaskResult with status, output paths, warnings, errors;
- validate that task_type is one of the supported values for Month 1: `
  run_golden_example`;
- add tests for valid and invalid task types.

Do not implement LangGraph yet.
```

7.4 Week 4 任务 2: 最小 Analysis Engineer wrapper

创建:

```
src/lqcd_agent/analysis_engineer/run.py
```

Analysis Engineer wrapper 的第一版职责:

1. 读取 AnalysisEngineerTask;
2. 检查 task type;
3. 调用 run_golden CLI 或内部函数;
4. 检查输出文件是否存在;
5. 写 AnalysisEngineerTaskResult;
6. 不做 final physics decision。

建议命令:

```
python -m lqcd_agent.worker.run_worker --task tasks/month1_golden_task.yaml
```

Codex prompt 示例:

```
Please implement a minimal Analysis Engineer wrapper for Month 1.

File:
`src/lqcd_agent/analysis_engineer/run.py`

Requirements:
```

```
- command: `python -m lqcd_agent.worker.run_worker --task tasks/month1_golden_task.
  yaml`;
- load AnalysisEngineerTask schema;
- support task_type = `run_golden_example`;
- call the existing golden example CLI or internal run function;
- check that result.json, summary.csv, manifest.json, and run.log exist;
- write `analysis_engineer_result.json`;
- if any required output is missing, mark status as failed and record errors;
- do not decide central physics results;
- do not overwrite existing outputs unless allow_overwrite is true;
- add pytest tests for success and missing config failure.
```

验收标准：

- Analysis Engineer task 可以运行 golden example;
- worker_result.json 生成;
- 输出缺失时 Analysis Engineer 能报告失败;
- Analysis Engineer 不做 fit selection 或 final physics decision;
- 测试通过。

7.5 Week 4 任务 3: Month 1 final report

学生需要写：

```
docs/month1_final_report.md
```

报告内容：

1. 项目目标理解;
2. golden example 复现结果;
3. data-flow map;
4. config schema 设计;
5. result schema 设计;
6. data audit;
7. pytest 测试列表;
8. Analysis Engineer wrapper 雏形;
9. 当前限制;
10. Month 2 计划。

7.6 Week 4 命令示例

```
# 一键运行 Analysis Engineer
python -m lqcd_agent.analysis_engineer.run \
  --task configs/tasks/month1_golden_task.yaml
# 检查输出
ls outputs/l64c64a076_m140/
cat outputs/l64c64a076_m140/correlator_diagnostics.json
# 运行全部测试
pytest -q
```

7.7 Week 4 常见错误

- 把 Analysis Engineer 当成物理判断者：它只执行和检查，不能决定 central fit。
- 输出没有 provenance：图和数值如果没有 config、git commit、run id 和 log，无法用于严肃科研。
- Month 1 final report 只列做了什么，不写学到什么：应该包含 failures 和 lessons。

7.8 Week 4 导师检查问题

- 是否能从 clean repo 一键运行 Month 1 全流程？
- pytest 是否全部通过？学生是否能解释每个测试保护什么？
- 学生能否解释 Analysis Engineer 的作用和边界？
- Month 1 report 是否包含复现命令、输出文件列表、failures 和 lessons？

7.9 Week 4 任务 4: Month 1 demo

学生应准备一个 10–15 分钟展示，内容包括：

- 一页说明项目目标；
- 一页说明 golden example 流程；
- 一页说明 repo 结构；
- 一页展示 CLI 运行命令；
- 一页展示 result.json / summary.csv / plots；
- 一页展示 pytest 结果；
- 一页说明 Analysis Engineer wrapper；
- 一页说明当前问题和 Month 2 计划。

8 第一个月导师验收清单

导师可以按以下清单检查。

8.1 复现性

- 是否能从 config 一键运行 golden example?
- 是否记录 git commit 和环境?
- 是否有 manifest.json?
- 是否有 run.log?
- README 是否足够另一个学生复现?

8.2 代码质量

- 是否有清楚的目录结构?
- 是否把硬编码路径移入 config?
- 是否函数边界清楚?
- 是否避免大范围无关修改?
- 是否避免修改 raw data?

8.3 测试

- pytest 是否通过?
- 测试是否覆盖 schema、data audit、CLI smoke test?
- 测试是否依赖机器特定路径? 如果依赖, 需要改掉。

8.4 学生理解

学生应能回答:

1. 什么是 golden example?
2. 为什么需要 config-driven pipeline?
3. manifest.json 和 result.json 有什么区别?
4. data audit 模块为什么不能修改 raw data?
5. Analysis Engineer 为什么不能决定 central physics result?
6. pytest 在科研分析中保护了什么?
7. Codex 写的代码为什么不能直接相信?

9 第一个月常见错误

9.1 错误 1: 一开始就让 Codex 写完整系统

这会导致代码过大、不可控、学生无法理解。正确做法是每天一个小任务。

9.2 错误 2: 只跑 notebook, 不记录命令

科研 agent 系统必须可复现。没有命令记录和 config, 就无法追责。

9.3 错误 3: 没有 schema

没有 schema, config 和 result 会逐渐变成不稳定的自由字典, 后续 agent 很难可靠读取。

9.4 错误 4: 没有 tests

没有 tests, Codex 每次修改都可能破坏已有功能。

9.5 错误 5: 把 Analysis Engineer 当成物理判断者

Analysis Engineer 可以运行和整理结果, 但不能决定 central fit、丢弃数据或给出最终物理结论。

9.6 错误 6: 输出没有 provenance

图和数值如果没有 config、git commit、run id 和日志, 就很难用于严肃科研。

10 Month 2 的衔接

第一个月结束后, Month 2 将进入更高级阶段:

1. 将 Month 1 的 CLI 包装进 Snakemake workflow;
2. 增加 Fit Strategy Postdoc、Matrix Element Analyst 和 Critical Referee;
3. 建立 fit candidate summary table;
4. 实现 rule-based Analysis Coordinator;
5. 写 decision_log.json;
6. 建立 accepted/rejected case memory.

所以 Month 1 的关键不是把所有高级功能做完, 而是打好以下基础:

```
config -> schema -> CLI -> audit -> output -> tests -> Analysis Engineer wrapper
```

只有这条链路清楚, Month 2 的 Snakemake 和 Analysis Coordinator 才能自然接上。

11 附录：第一个月推荐 Codex Prompt 模板

11.1 通用模板

We are building an AI-assisted LQCD meson DA analysis-agent system.

Current stage: Month 1 foundation.

Task:

[describe one small task]

Files to edit:

[list files]

Requirements:

- [requirement 1]
- [requirement 2]
- [requirement 3]

Inputs:

[describe inputs]

Outputs:

[describe outputs]

Tests:

Please add pytest tests for:

- [test 1]
- [test 2]

Constraints:

- Do not modify raw data.
- Do not modify unrelated files.
- Do not make final physics decisions.
- Do not overwrite existing outputs unless explicitly allowed.
- Keep the implementation simple and readable.

After implementation, explain:

- what changed;
- how to run it;
- what tests were added.

11.2 修 bug 模板

The following command failed:

```
[command]

Traceback:
[paste traceback]

Expected behavior:
[expected]

Actual behavior:
[actual]

Please fix only the cause of this failure.
Do not refactor unrelated code.
If appropriate, add a regression test so this failure is caught in the future.
```

11.3 让 Codex 加测试的模板

```
Please add pytest tests for the existing implementation.

Focus on:
- valid input case;
- invalid input case;
- missing file or missing directory case;
- output schema validation;
- CLI smoke test using tmp_path.

Do not change the implementation unless a test reveals a real bug.
If you find a bug, explain it first and then make a minimal fix.
```

12 第一个月最终标准

第一个月完成后，学生应该达到以下水平：

- 能复现导师给定的 golden example；
- 能将手动分析流程拆成输入、处理、输出和检查；
- 能写清楚 config schema 和 result schema；
- 能用 Codex 辅助生成代码，但知道如何检查和修正；
- 能写最小 pytest；
- 能用 Git 做小步提交；

- 能解释 Analysis Engineer 的作用和边界；
- 能用一个命令运行基础 pipeline；
- 能提交一份清楚的 Month 1 final report。

如果学生完成以上内容，就已经具备继续进入 Month 2 高级 workflow 和 Analysis Coordinator agent 阶段的基础。